

Lenguajes y Gramaticas Formales



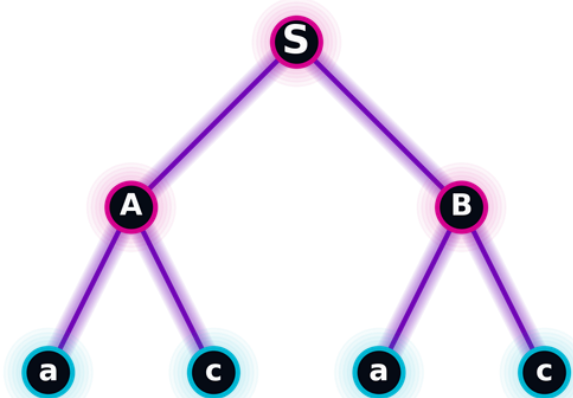
Fundamentos y Jerarquía de Chomsky

Análisis de la Jerarquía de Chomsky. Exploramos cómo las reglas gramaticales definen la capacidad de reconocimiento de un lenguaje, proporcionando las bases necesarias para entender la relación entre gramáticas formales y los autómatas que las procesan.





Relación Gramática-Lenguaje:



Generación de Cadenas

Una gramática formal es un conjunto de reglas matemáticas que define cómo estructurar y generar cadenas de caracteres válidas para un lenguaje determinado. El mecanismo mediante el cual una gramática genera un lenguaje consiste en partir del símbolo inicial y aplicar sucesivamente las reglas de producción hasta obtener una cadena compuesta únicamente por símbolos terminales. El conjunto de todas las cadenas que pueden derivarse de esta manera constituye el lenguaje generado por la gramática.

Jerarquía de Chomsky (4 niveles):

La Jerarquía de Chomsky clasifica las gramáticas formales en cuatro tipos, donde cada tipo añade restricciones al anterior y está asociado a un autómata específico.

Nivel 0

El Tipo 0, conocido como gramáticas sin restricciones, es el nivel más general. Sus reglas de producción no tienen limitaciones y pueden describir cualquier lenguaje computable.

Nivel 1

Impone la restricción de que las reglas de sustitución dependen de los símbolos que rodean a la variable.

Nivel 2

es el nivel donde las variables se reemplazan de forma independiente a su entorno, sin importar los símbolos que las rodean.

Nivel 3

Conocido como gramáticas regulares, es el nivel más simple y restrictivo. Sus reglas son lineales, del tipo $A \rightarrow aB$ o $A \rightarrow a$, lo que limita su capacidad a patrones simples.

Derivación y Modelado (Genoma)

Explorando la intersección entre la teoría de lenguajes formales y el diseño gráfico. Esta propuesta utiliza una Gramática Libre de Contexto para codificar formas geométricas y estructuras orgánicas en cadenas de 'genoma visual', demostrando cómo reglas simples pueden generar una complejidad infinita mediante la iteración y la recursión.



Gramática Libre de Contexto para Dibujo Genético

GLC inspirada en L-systems que modela figuras geométricas y biológicas con alfabeto $\Sigma = \{a, c, g, t\}$ e interpretación mediante modelo de tortuga.

Correspondencia de comandos:

Símbolo	Comando	Significado
a	Avanza dibujando	Trazo recto
c	Cambia dirección	Giro (90°, 60° o 120°)
g	Guarda estado	Push (posición y orientación)
t	Recupera estado	Pop (restaura)

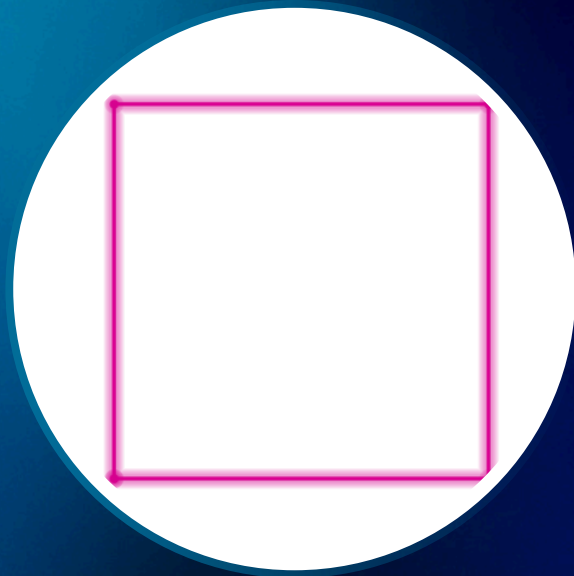
Definición formal:

- No terminales: $N = \{D, F, R\}$
- Símbolo inicial: D
- Producciones:
 - $D \rightarrow F \mid F R D$
 - $F \rightarrow a F \mid c F \mid \varepsilon$
 - $R \rightarrow g F t R \mid \varepsilon$

Derivación Paso a Paso – Figuras Geométricas

1. Cuadrado (ángulo 90°):

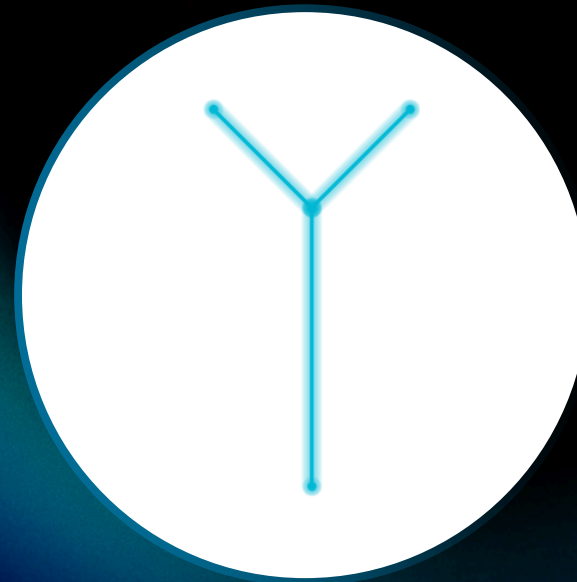
$D \Rightarrow a c a c a c a$



4 lados con giros de 90°
que cierran la figura.

2. Árbol con Ramas (ángulo 45°):

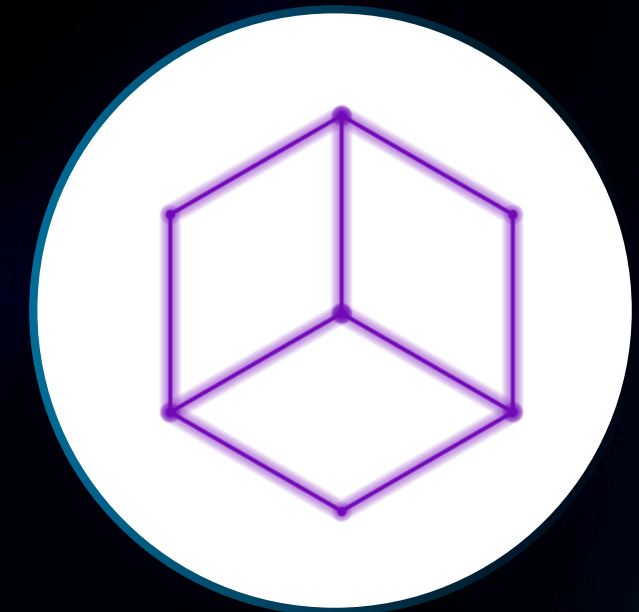
$D \Rightarrow a a g a t g a t$



Tronco de 2 unidades, guarda
estado, dibuja rama izquierda,
recupera, dibuja rama derecha.
Forma de "Y".

3. Cubo (proyección isométrica, ángulo 120°):

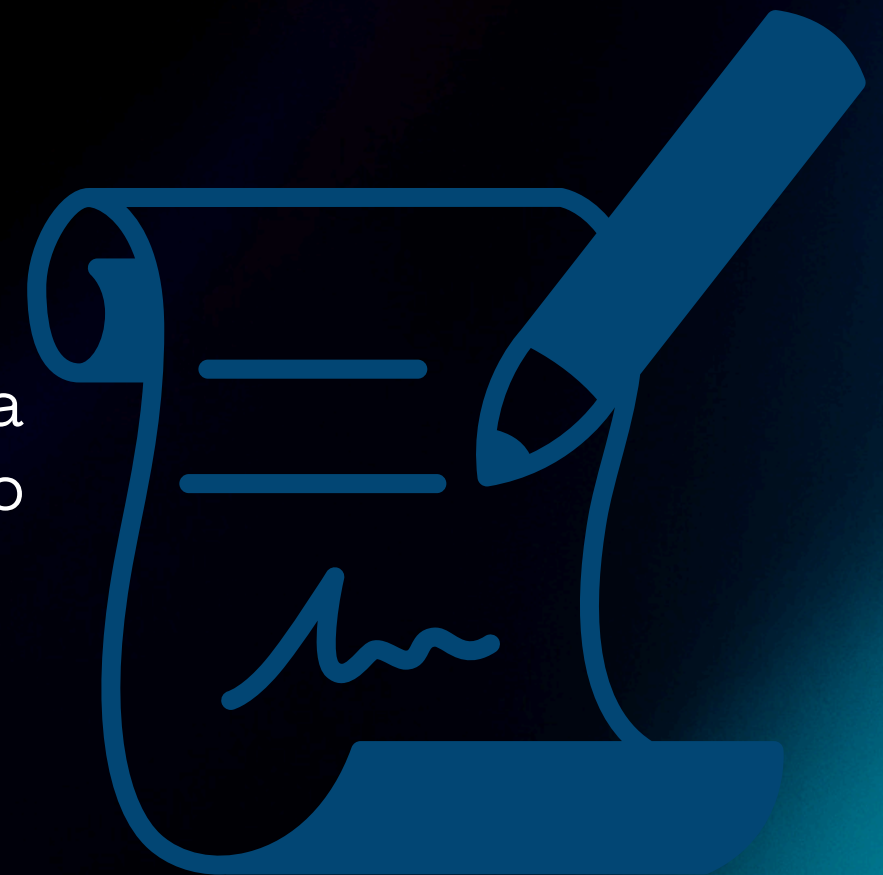
$D \Rightarrow a c a c a c a c c c a c a c$
 $a c a$



Cuadrado base $\rightarrow$ cambio de plano $\rightarrow$
arista vertical $\rightarrow$ segundo cuadrado
desplazado. Ilusión de cubo en 2D.

Higiene y Optimización de Gramáticas

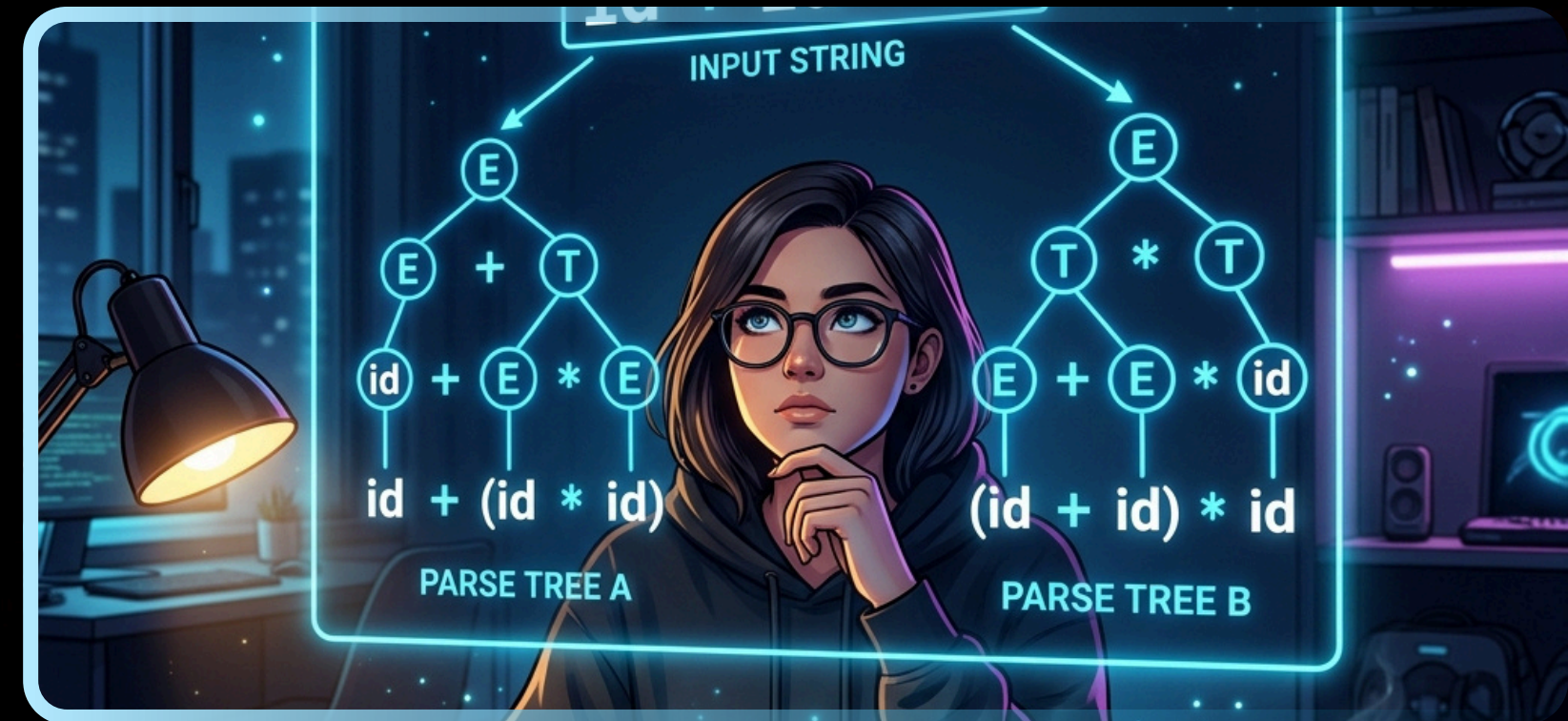
Este apartado se centra en la aplicación de técnicas de optimización y limpieza (higiene) de gramáticas, fundamentales para garantizar la eficiencia en el modelado de figuras geométricas y estructuras biológicas tipo L-systems.



Ambigüedad

Una gramática es ambigua si una misma cadena puede generarse con dos o más árboles de derivación distintos.

- Una misma cadena puede tener ≥ 2 árboles de derivación.



Ejemplo clásico:

$$E \rightarrow E + E \mid E * E \mid \text{num}$$


Corrección:

Jerarquía de operadores y reglas no ambiguas.

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow \text{num} \end{aligned}$$

Ahora la multiplicación tiene prioridad y la cadena tiene una única derivación válida.

Recursividad por la Izquierda

Una regla de la forma $A \rightarrow A\alpha \mid \beta$ donde A se llama a sí misma como primer símbolo.

Problema:

En analizadores sintácticos descendentes (predictivos), esto genera un bucle infinito:

$A \rightarrow A \rightarrow A \rightarrow \dots$

Ejemplo:

$A \rightarrow Aa \mid b$

Algoritmo de eliminación (paso a paso):

1. Identificar la regla recursiva por izquierda.

2. Reescribir como:

$A \rightarrow \beta A'$
 $A' \rightarrow \alpha A' \mid \epsilon$

Corrección aplicada:

$A \rightarrow bA'A' \rightarrow aA' \mid \epsilon$

La recursión ahora es derecha, controlable y no infinita.

Falta de Factorización por la Izquierda



Dificulta la elección de producción en parsers predictivos

Algoritmo de factorización

1. Extraer prefijo común α .
2. Crear nueva regla:

$A \rightarrow \alpha A'$
 $A \rightarrow \beta_1 \mid \beta_2$

Corrección



Antes: $S \rightarrow \text{if } E \text{ then } S \text{ else } S \mid \text{if } E \text{ then } S$
Después: $S \rightarrow \text{if } E \text{ then } S S'$
 $S' \rightarrow \text{else } S \mid \epsilon$

Ahora el analizador lee el prefijo común sin dudas y decide después con S' .

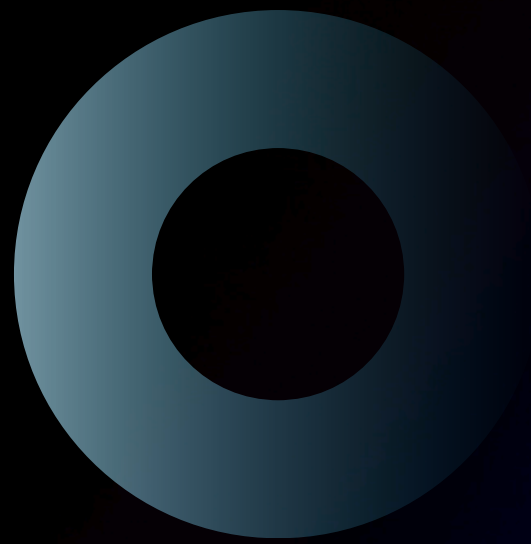
De la Expresion Al Automata ***(Ajedrez PGN)***

El objetivo es explicar la transformación de una notación textual estructurada (el formato PGN de ajedrez) hacia un modelo computacional ejecutable (un autómata).



presented by: **OLIVIA WILSON**

El subconjunto de movimientos elegidos



- 1. Movimientos de peón y piezas (ej. e4, Nf3).
- 2. Capturas (ej. exd5, Nxf3).
- 3. Desambiguación de origen (ej. Nbd2, R1e4).
- 4. Enroques corto (O-O) y largo (O-O-O).
- 5. Desambiguación de origen (ej. Nbd2, R1e4).
- 6. Jaques (+) y mate (#) opcionales al final.



La Expresión Regular Diseñada

Tenemos un gran bloque dividido por un "O" lógico (|): La primera mitad valida movimientos normales y la segunda mitad valida enroques (O-O(-O)?).

```
# Subconjunto cubierto:
# [KQRBN]?      pieza opcional (ausencia → peón)
# [a-h]?        columna de origen desambiguación
# [1-8]?        fila de origen desambiguación
# x?            captura opcional
# [a-h]         columna destino (obligatoria)
# [1-8]         fila destino (obligatoria)
# ([QRBN])?     promoción opcional
# | O-O(-O)?    enroque corto o largo
# [+#]?        jaque o jaque mate opcional
REGEX_PGN = (
    r"^[("
    r"[KQRBN]?[a-h]?[1-8]?x?[a-h][1-8](=[QRBN])?" # movimiento normal
    r"|"
```

[KQRBN]?:
Busca la pieza inicial (si no hay, asume que es un peón).

[a-h][1-8]:
Es el corazón de la expresión, la casilla de destino obligatoria.

[+#]?:
Revisa si hay una promoción opcional.

[a-h]?[1-8]?:
Es la desambiguación opcional (columna o fila de origen).

x?:
Revisa si hubo captura.

(=[QRBN])?:
Revisa si hay una promoción opcional.

Cómo funciona el Autómata Finito Determinístico (AFD)



Para procesar esto eficientemente, el Programa 3 implementa un Autómata Finito Determinístico, es decir, una máquina de estados que lee el texto carácter por carácter y lo clasifica en categorías (Pieza, Columna, Fila, Captura, etc.)."

Nuestro sistema implementa un Autómata Finito Determinista (AFD) diseñado para validar la sintaxis del formato PGN. Consta de 14 estados principales (q0 a q13) que representan las fases del análisis y un Estado Trampa (o estado de rechazo)

Gracias!

